Modern OS Concepts for Devs

The road from localhost to production

Threads vs Processes



Mindset Shift

- **“Competition is the easy part. Behind the scenes is where all the work is done.” – Usain Bolt**



Localhost app



Localhost vs Production app



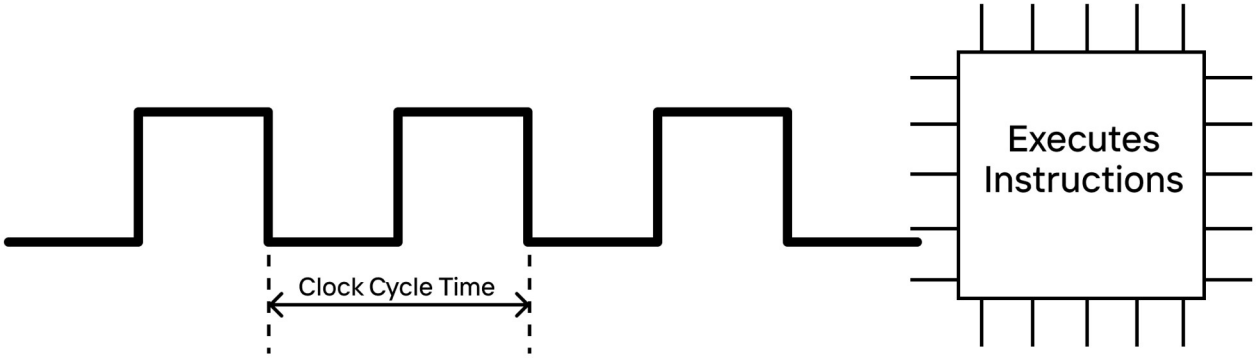


We build systems for production



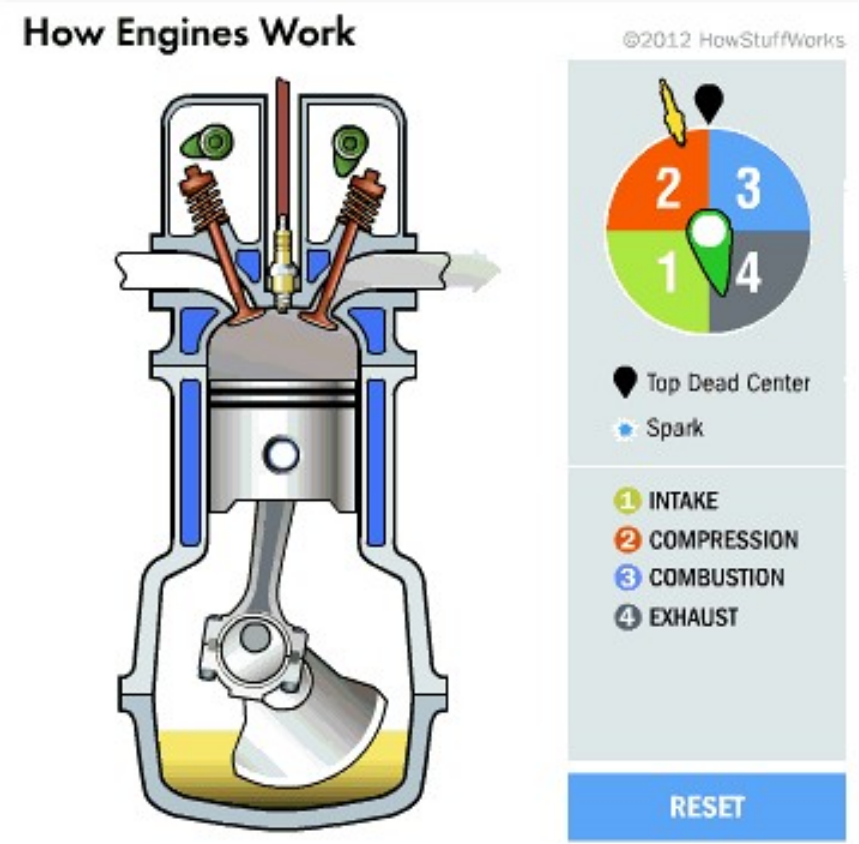
CPU



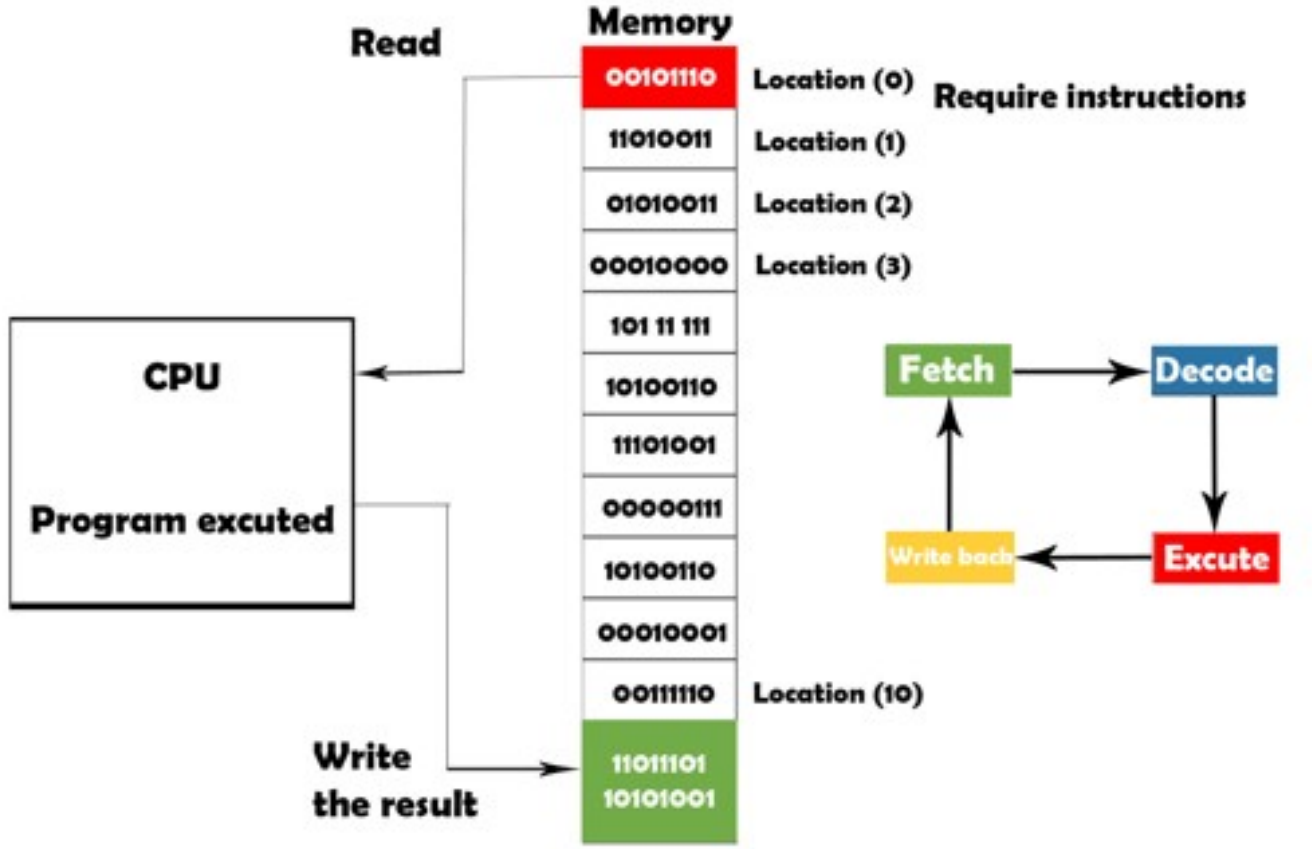


CPU Time = Instruction Count × Cycles Per Instruction × Clock Cycle Time

Engine


How Engines Work
©2012 HowStuffWorks
2
3
1
4
Top Dead Center
Spark
1 INTAKE
2 COMPRESSION
3 COMBUSTION
4 EXHAUST
RESET

CPU Cycle


Read
Memory
00101110
Location (0)
Require instructions
11010011
Location (1)
01010011
Location (2)
00010000
Location (3)
101 11 111
10100110
11101001
00000111
10100110
00010001
00111110
Location (10)
11011101
10101001
CPU
Program excuted
Write
the result
Fetch
Decode
Excute
Write back

Early x86

- **Had no concept of threading**
- **programs could access any memory**
  - No memory protection
  - No isolation between programs
  - Memory is just a flat, shared space
- **So any program could:**
  - Read/write any memory address
  - Overwrite another program
  - Even overwrite the OS itself

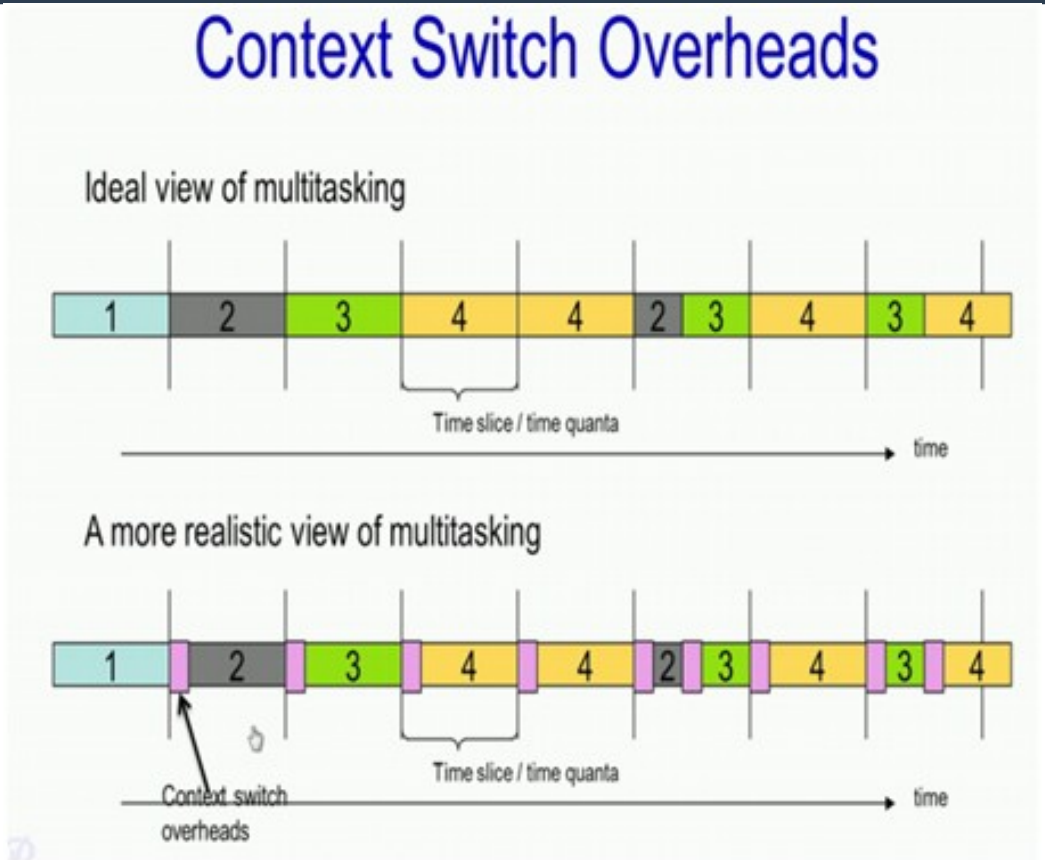
Why it was designed like this?

- Ran one program at a time (or very simple DOS-style multitasking)
- Prioritized simplicity and performance over safety

How engineers simulate threading?

- **Software threads came first**
- **Even before hardware support:**
  - OSes (like early Unix, Windows) introduced threads as a software abstraction
  - The CPU still ran one thread at a time, just switching very fast

Context Switching



When this changed?

- **With Intel 80386**
- **we got:**
  - Protected mode
  - Memory isolation per process

Modern x86 (hardware threads exist)

- **Starting in the early 2000s, CPUs added Simultaneous Multithreading (SMT):**
  - Intel Pentium 4 introduced Hyper-Threading
  - Modern CPUs like Intel Core i7 or AMD Ryzen support multiple threads per core
- **This means:**
  - One physical core can execute multiple threads at the same time
  - Example: 4 cores + 2 threads per core = 8 threads
  - Hyper-Threading enabled → 8 threads visible to the OS
  - Task Manager shows 8 CPUs, but physically there are only 4 cores.

Software & Hardware evolve together

- **Hardware**
  - Memory model (real mode vs protected mode)
  - Hardware threads
  - Multiple cores
- **Software**
  - The OS schedules threads (Software threads)
  - Virtual Memory
  - Kernel space vs User space
- **Together**
  - Modern OSes rely heavily on hardware-enforced isolation
  - Drivers & Controllers
  - Hardware adds features → software adapts
  - Software demands features → hardware evolves

As a developer

- **You should think of Thread as a software thread**
- **CPU is not a single-user resource, it is time-shared**
- **Memory is a scarce resource, ignoring limits leads to:**
  - Memory
  - OOM kills in containers
  - Latency spikes from swapping
  - Unpredictable scaling issues in production

Processes & Threads

- **Program: a sequence of instructions, compile/interpreted**
- **Process is an instance of a program that is loaded into memory and being executed.**

Resources required by Process

- **Data Segment**
- **Code Segment**
- **Heap (Dynamic memory allocation)**
- **Stack (local variables, function parameters and calls)**
- **Registers (fast storage locations directly in CPU)**
  - Hold data temporarily during execution including
    - Stack Pointer → Points to the top of the stack in memory.
    - Program Pointer (Instruction Pointer) → holds the address of the next instruction the CPU will execute.

Assembly code example

```
section .data
    num1 db 5
    num2 db 3
    result db 0

section .text
    global _start

_start:
    mov al, [num1]   ; load num1 into AL register
    add al, [num2]   ; add num2 to AL
    mov [result], al ; store result

    ; exit program
    mov eax, 60      ; syscall: exit
    xor edi, edi     ; status = 0
    syscall
```

How OS creates a process

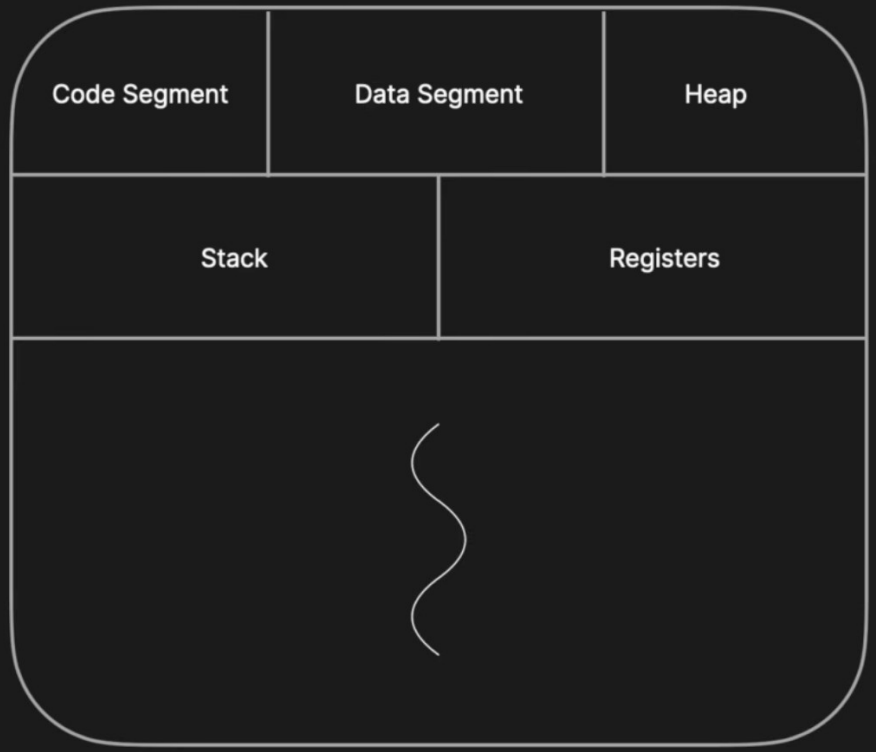
```
pub struct ProcessControlBlock {
    pid: u16,
    state: ProcessState,
    program_counter: u16,
    general_purpose_registers: [u8; 4],
    instruction_register: u8,
    flags: [u1; 3],
    stack_pointer: u16,
    index_registers: [u16; 2],
    memory_limits: [u16; 2],
    io_devices: Vec<IO_Device>,
    open_files: Vec<File>,
    parent: Arc<ProcessControlBlock>,
}
```

Process



- **Process is an instance of a program that is loaded into memory and being executed.**
- **Separate memory space**
  - One process can’t corrupt another process

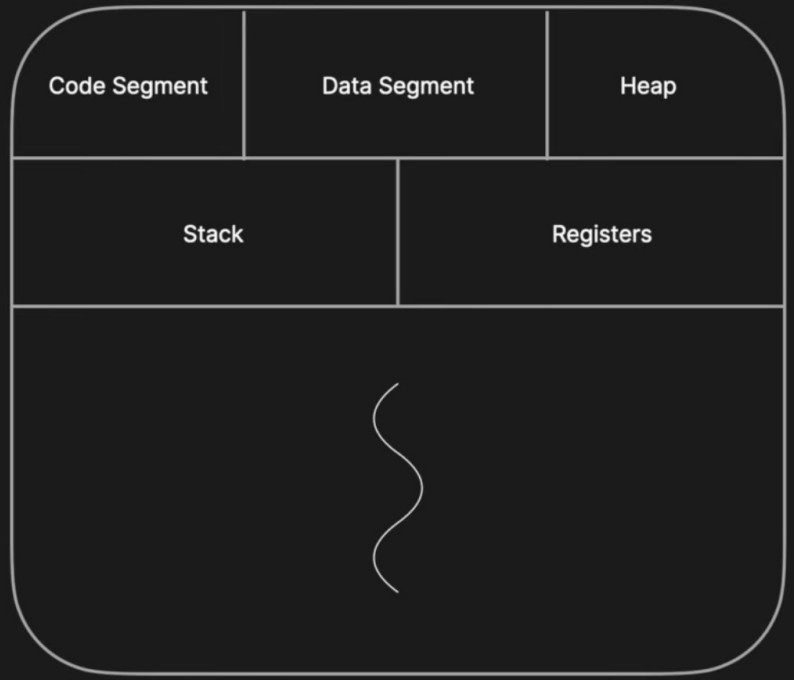
Thread

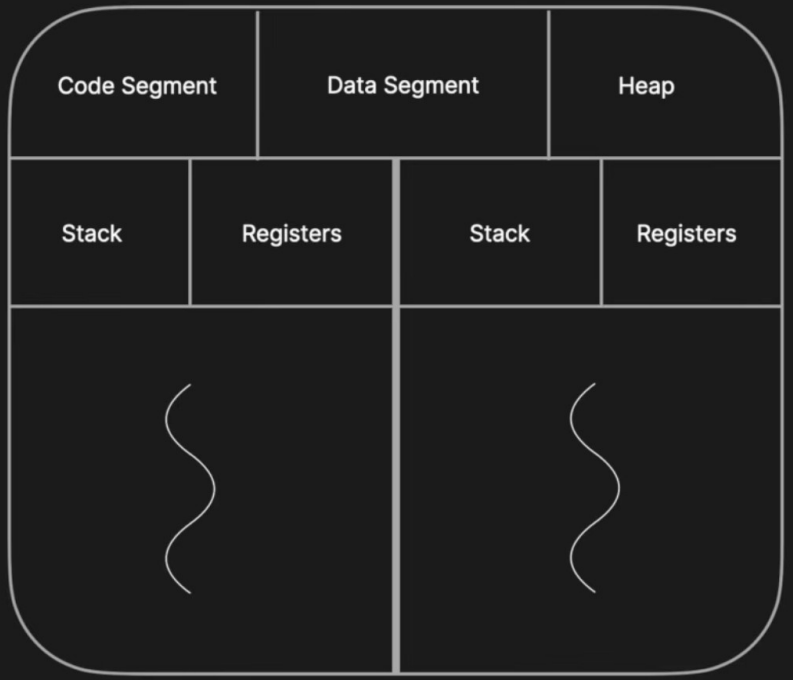


A unit of execution within a process

Unit → the smallest piece that can run independently

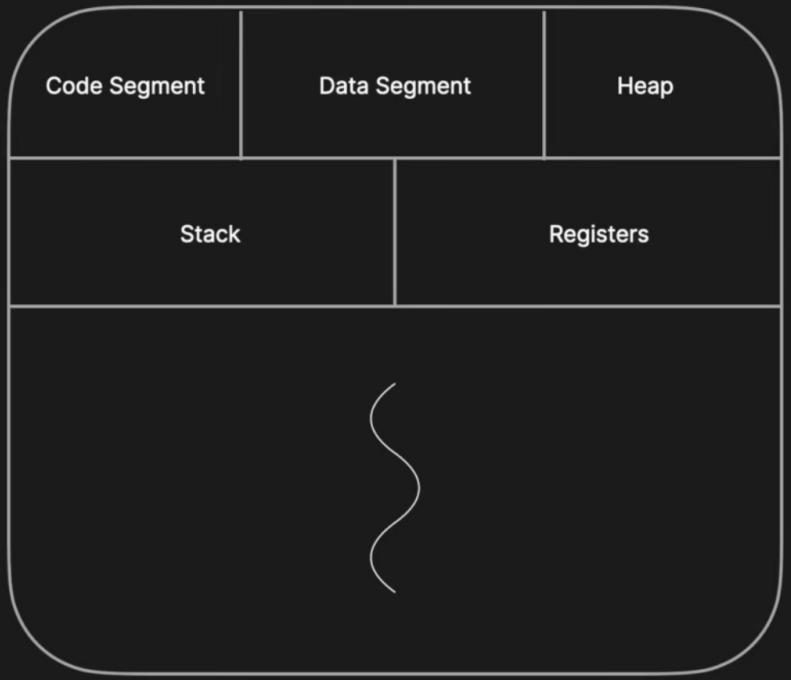
Single threaded vs Multi threaded process

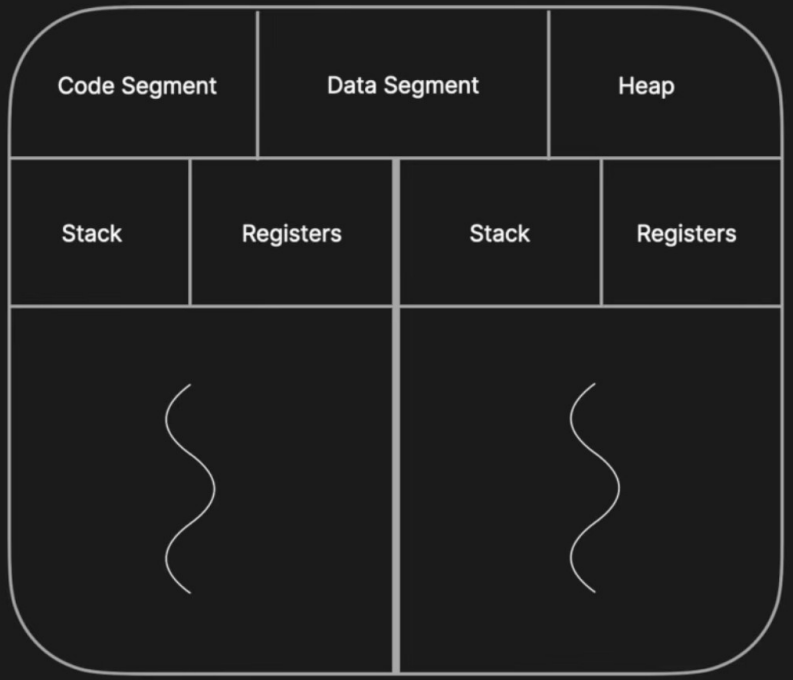




Single threaded vs Multi threaded process

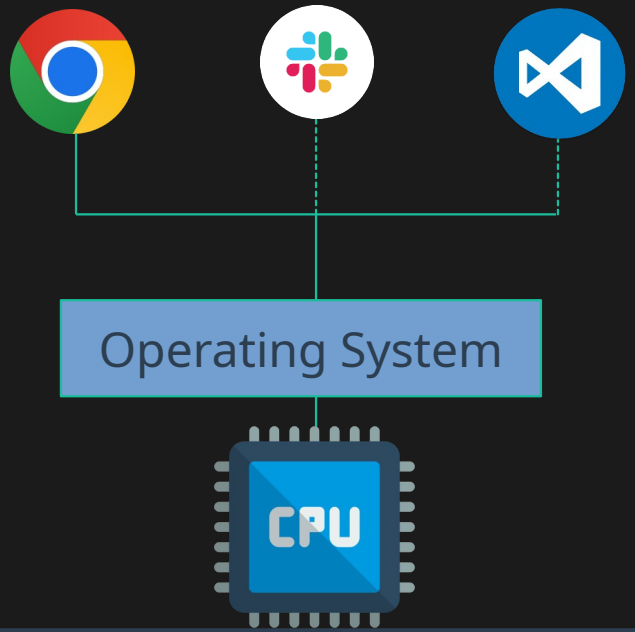
- Efficient communication between threads
- A single thread can corrupt the entire process





Process

- While process is a program in execution
- It doesn’t necessarily mean that the process is always using the CPU
- Even if CPU is allocated to that process



Threads with Context switching

PC=3

```
1  function hello(){
2      instruction1
3      instruction2
4      instruction3
5      instruction4
6      instruction5
7  }
```

PC=3

```
1  function hello(){
2      instruction1
3      instruction2
4      instruction3
5      instruction4
6      instruction5
7  }
```


Thank you